Databricks Level 7 (Governance, Security & Lineage) — Building Data Platforms That Are Secure, Auditable, and Compliant by Design



Summary

Databricks Level 7 (Governance, Security & Lineage) is where a platform that is fast, cheap and observable becomes one an auditor, a security team and a regulator can trust.

Level 5 (Production Readiness & Observability) told you whether the data was healthy. Level 6 (Performance & Cost Optimisation) told you what it cost.

Level 7 answers three questions:

1. Who can access what?
2. Where did this data come from, and where did it go?
3. Can we prove it?

This blog covers:

- The Unity Catalog object model and privilege design
- Groups, service principals and ownership
- Row filters and column masks
- ABAC with governed tags
- PII classification
- Lineage for debugging and compliance
- Audit logging
- GDPR “right to be forgotten” on Delta tables
- Secure sharing
- Governance operating models

The theme is governance by default: controls that apply automatically to new data, instead of a spreadsheet someone updates after the fact.

. . .

Who Should Read This Blog

1. **Data engineers whose tables now contain customer PII, financial or health data**
2. **Platform and security teams designing access models across many workspaces**
3. **Engineers working for European employers where GDPR is a daily concern**
4. **Anyone preparing for senior or staff-level interviews, where governance is a standard design topic**
5. **Tech leads who have been asked “who has access to this table?” and couldn’t answer in under an hour**
6. **Teams preparing for an audit (SOC 2, ISO 27001, GDPR)**

7. **Anyone who has ever shared a spreadsheet export "just this once"**

. . .

The Questions That Arrive After the Pipeline Is Perfect

Your pipelines are reliable (Level 4 – Pipeline Engineering). You can see them (Level 5 – Production & Collaboration). They are fast and affordable (Level 6 – Cost Optimisation). Then these questions show up:

- "A customer asked us to delete all their data. Did we?"
- "Which dashboards break if we drop this column?"
- "Why can the marketing intern see salary data?"
- "Who queried the customer table last Tuesday at 2 AM?"
- "Can a partner see only their own rows, without us building a copy for every partner?"

None of these are performance problems. They are trust problems, and they are expensive, because the answer is often "we'd have to investigate."

Level 7 is the discipline of making those answers a query, not a project.

. . .

The Mindset: Governance Is a Product, Not a Gate

Bad governance is a ticket queue where every access request takes a week. Teams then work around it with exports, copies and shared credentials, which is less secure than no governance at all.

Good governance is:

- Secure by default: new data is protected without anyone remembering to protect it
- Self-service where safe: analysts get the data they're entitled to without filing tickets
- Auditable: every access and change leaves a trail
- Proportionate: controls match the sensitivity of the data

Three principles run through everything below:

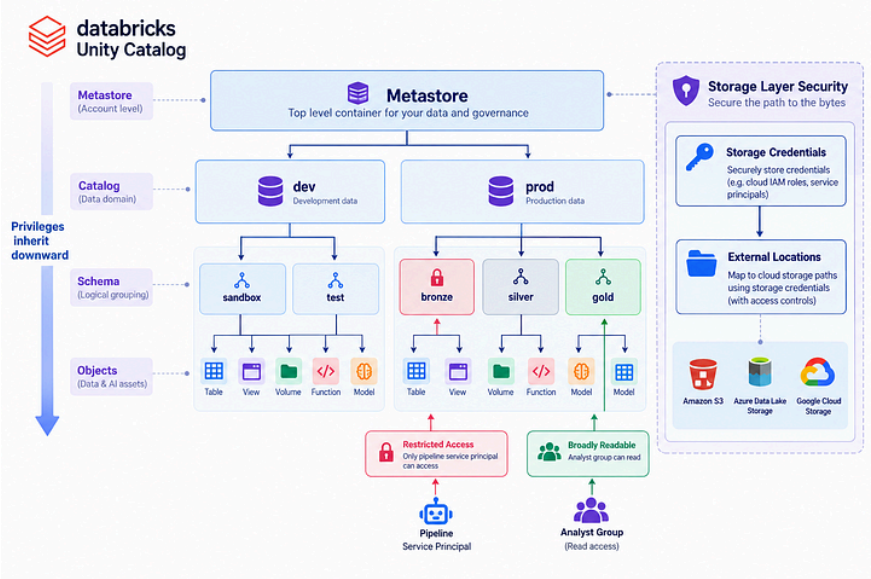
Principle	Meaning
Least privilege	Grant the minimum access needed, to groups, not individuals
Centralize policy, distribute ownership	One consistent policy model, with domain teams owning their data
Prove, don't assume	If you can't query evidence of a control, treat it as not existing

Step 1: The Unity Catalog Object Model

Everything in Unity Catalog hangs off one hierarchy:

```
Metastore
└── Catalog                (e.g., prod, dev)
    └── Schema             (e.g., silver, gold)
        ├── Table / View / Materialized View
        ├── Volume         (files)
        ├── Function       (including mask and fil
        └── Model
```

Privileges inherit downward. A grant on a schema applies to the objects in it, including ones created later. That inheritance is your main design tool.



Using the hierarchy as a security boundary

A common layout that scales well:

Pattern	Example	Why
Catalog per environment	`dev`, `staging`, `prod`	Separate blast radius and permissions
Schema per layer or domain	`prod.bronze`, `prod.silver`, `prod.gold`	Different audiences per layer
Catalog per domain (large orgs)	`sales_prod`, `finance_prod`	Domain ownership and isolation

Raw layers (Bronze) usually contain the most sensitive and least-curated data. Restrict them to the pipeline's service principal and a small engineering group. Analysts read Gold.

Securable objects beyond tables

Don't forget what sits underneath:

- Storage credentials and external locations control access to cloud storage paths. Restrict who can create and use them, or people can bypass table-level governance by reading files directly.

- Volumes govern non-tabular files.
- Connections govern federated sources.

Table-level security means little if people can read the underlying storage directly. Governance has to cover the path to the bytes, not just the table name.

. . .

Step 2: Identities—Groups, Service Principals and Ownership

Grant to groups, never to individuals

When someone changes teams, you remove them from a group. You don't hunt for individual grants across hundreds of tables.

```
analysts_emea        → read Gold data for EMEA
analysts_global      → read Gold data for all re
pii_readers          → may see unmasked PII
data_engineers       → write to Silver/Gold in d
platform_admins      → manage infrastructure
```

Sync groups from your identity provider (Entra ID, Okta and similar) using SCIM, so the identity system stays the source of truth.

Service principals for automation

Pipelines should run as service principals, not as a person's account.

- Jobs don't break when someone leaves
- Audit logs show clearly "this was the pipeline," not "Anurag at 3 AM"
- Permissions are scoped to what the pipeline needs

This connects to Level 4: run production jobs as a service principal and keep humans out of production write paths.

Ownership matters more than people expect

Every securable object has an owner, and owners have broad control, including granting access to others. Set ownership to a group or service principal, not an individual, so that the object doesn't become orphaned when someone leaves.

A least-privilege grant example

```
-- Analysts can discover and read curated Gold d
GRANT USE CATALOG ON CATALOG prod TO `analysts_g
GRANT USE SCHEMA  ON SCHEMA  prod.gold TO `analy
GRANT SELECT      ON SCHEMA  prod.gold TO `analy

-- The pipeline's service principal can write Si
GRANT USE CATALOG ON CATALOG prod TO `sp_silver_
GRANT USE SCHEMA, SELECT, MODIFY, CREATE TABLE
  ON SCHEMA prod.silver TO `sp_silver_pipeline`;
```

Notes:

- A user needs `USE CATALOG` and `USE SCHEMA` along the path, plus `SELECT` on the object (or on the schema or catalog above it, by inheritance).
- Avoid `ALL PRIVILEGES` grants except for tightly controlled admin groups.
- Grant at the schema level where possible. Granting per table scales poorly.

Check your current privilege names against the Unity Catalog privileges reference, because the set has grown over time.

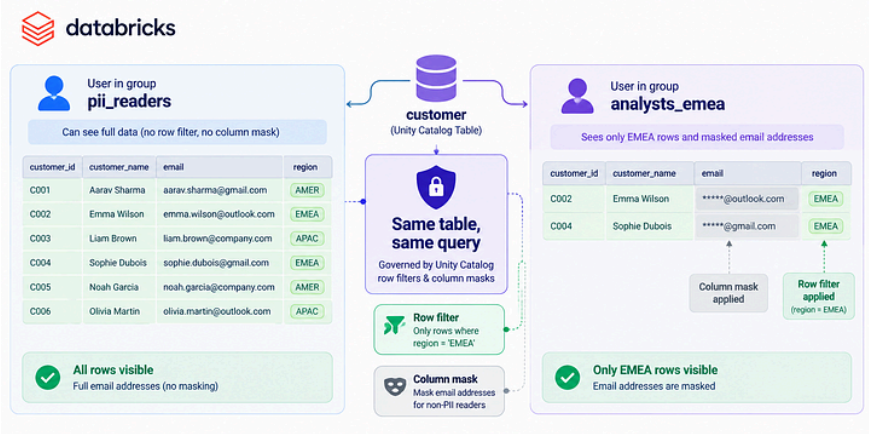
. . .

Step 3: Row Filters and Column Masks — Fine-Grained Access Control

Table-level access is too coarse for real data. The same `customers` table might need:

- Row-level security: an EMEA analyst sees only EMEA rows
- Column-level masking: most people see `j****@gmail.com`, and only a small group sees the real email

Unity Catalog supports both, and the logic lives with the data, so every tool that queries the table is covered.



Column mask, the table-level way

```
-- 1. A governed, reusable function
CREATE OR REPLACE FUNCTION gov.masks.mask_email(
RETURNS STRING
RETURN CASE
  WHEN is_account_group_member('pii_readers') TH
  ELSE regexp_replace(email, '^[^@]+', '*****')
END;

-- 2. Attach it to a column
```

```
ALTER TABLE prod.silver.customers
  ALTER COLUMN email SET MASK gov.masks.mask_ema
```

Now `SELECT email FROM prod.silver.customers` returns `*****@gmail.com` for everyone outside `pii_readers`.

Row filter, the table-level way

```
CREATE OR REPLACE FUNCTION gov.filters.region_fi
RETURNS BOOLEAN
RETURN
  is_account_group_member('analysts_global')
  OR (is_account_group_member('analysts_emea') A

ALTER TABLE prod.silver.orders
  SET ROW FILTER gov.filters.region_filter ON (r
```

Two analysts run the same query and see different rows, with no per-team copies of the table.

Why this beats the alternatives

Approach	Problem
Separate table copy per audience	Drift, storage cost, pipeline sprawl, forgotten refreshes
Dynamic views with `CASE` logic	Works, but logic is duplicated across views, and anyone with access to the base table bypasses it
Row filters and column masks on the table	One definition, enforced for every query path

Practical cautions

- Test with real identities. Verify as a member and a non-member of each group. Don’t trust the function until you’ve seen both outputs.

- Keep functions simple. Complex logic or lookups inside a mask function can hurt query performance. Level 6 applies here too.
- Mind the downstream. Masked values copied into a new table are no longer masked unless that table also has protection. Governance has to follow the data through your pipelines (this is where lineage helps, in Step 5).
- Check compute requirements. Fine-grained access control has requirements on compute type and runtime. Verify them in the docs for your environment before rollout.

. . .

Step 4: ABAC—Policies That Follow the Data

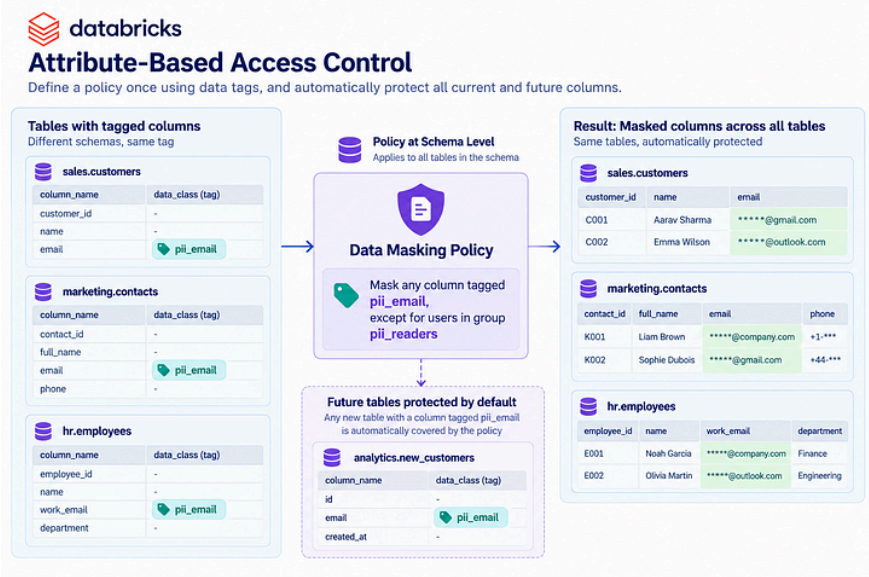
Table-level masks and filters have a scaling problem. If you have 400 tables with an `email` column, someone must attach the mask 400 times, and every new table needs the same step. This is exactly how governance decays.

Attribute-based access control (ABAC) in Unity Catalog flips the model. Per the Databricks docs, access is determined by evaluating attributes on objects, represented as governed tags, which policy conditions use to identify the data the policy should protect.

The core ideas

Concept	What it is
Governed tag	A tag with a controlled definition (who can create and assign it, and which values are allowed)
Policy	A rule attached at the catalog, schema or table level (metastore-level is in Beta per the docs)
Row filter policy	Filters rows when matching tags are present
Column mask policy	Masks columns carrying matching tags
GRANT / DENY policies	Dynamic privilege grants, and explicit denials (DENY is Beta)

Policies are evaluated dynamically. When a table or column acquires the targeted attribute, the policy takes effect automatically.



The workflow

1. Define a tag taxonomy: for example, `sensitivity = {public, internal, confidential, restricted}` and `data_class = {pii_email, pii_phone, financial}`.
2. Tag the data: columns holding emails get `data_class = pii_email`.
3. Write one policy per rule: “mask any column tagged `pii_email` for everyone except `pii_readers`."
4. Attach at the right scope: a catalog or schema.
5. Let it propagate: every current and future table in scope with that tag is protected.

What a column mask policy looks like

The exact SQL has evolved across releases, so treat this as a shape, and check the current syntax in the docs before using it:

```
-- Illustrative: verify exact syntax against cur
CREATE POLICY mask_email_columns
ON SCHEMA prod.silver
COLUMN MASK gov.masks.mask_email
```

```
  TO `account users`
  EXCEPT `pii_readers`
  FOR TABLES
  MATCH COLUMNS hasTagValue('data_class', 'pii_ema
  ON COLUMN c;
```

The structure to remember: who the policy applies to, who is excepted, what it matches (by tag), and which function does the masking.

When to use ABAC vs table-level filters and masks

Situation	Better fit
Many tables share the same sensitive column types	ABAC
You want new tables protected automatically	ABAC
Central security team sets rules; domain teams own data	ABAC
One special table with unique, bespoke logic	Table-level filter or mask
Need an override for a single table	Table-level (the docs cover override behavior)

The docs include a dedicated comparison page, “When to use ABAC vs table-level row filters and column masks,” which is worth reading before you commit.

Best practices

- Design the tag taxonomy first. A messy taxonomy produces messy policy. Keep it small and consistent.
- Control who can assign governed tags. If anyone can retag a column `public`, the policy is decorative.
- Start with a few high-value policies. PII masking and a regional row filter cover a lot of ground.
- Review the documented limitations. The docs list quotas, compute requirements and current gaps such as view support and conflict resolution. Check them for your case.

- Watch performance. Simple functions and predicate-friendly logic keep queries fast.

. . .

Step 5: Classification—Finding Sensitive Data Before It Finds You

You can't protect data you don't know you have. Classification is the step that feeds tags into your ABAC policies.

A pragmatic approach

1. Automated discovery: use the platform's data classification features where available, and complement them with pattern checks (emails, phone numbers, national IDs, card numbers). Verify current feature availability in your region and edition.
2. Human review: automated classifiers produce false positives and negatives. A data steward confirms the important ones.
3. Tag and enforce: confirmed classifications become governed tags, and policies take over.
4. Re-scan regularly: new columns appear all the time.

A simple classification scheme

Level	Examples	Typical controls
Public	Published product catalog	Open to all authenticated users
Internal	Aggregated sales by region	Employees only
Confidential	Customer names, order history	Role-based, audited
Restricted	National IDs, health data, card data	Masked by default, tightly limited, fully audited

Reduce what you store in the first place

The best control for sensitive data is not holding it.

- Drop fields you never use

- Tokenize or hash identifiers where analytics only needs consistency, not the real value
- Use surrogate keys (`user_id`) instead of natural identifiers (email) as join keys. This also makes deletion far easier (Step 8).

Be careful with hashing. Hashing low-entropy values like phone numbers or small ID ranges without a secret salt can be reversed by brute force. Treat that as pseudonymization, not anonymization.

. . .

Step 6: Lineage—Knowing Where Data Came From and Where It Goes

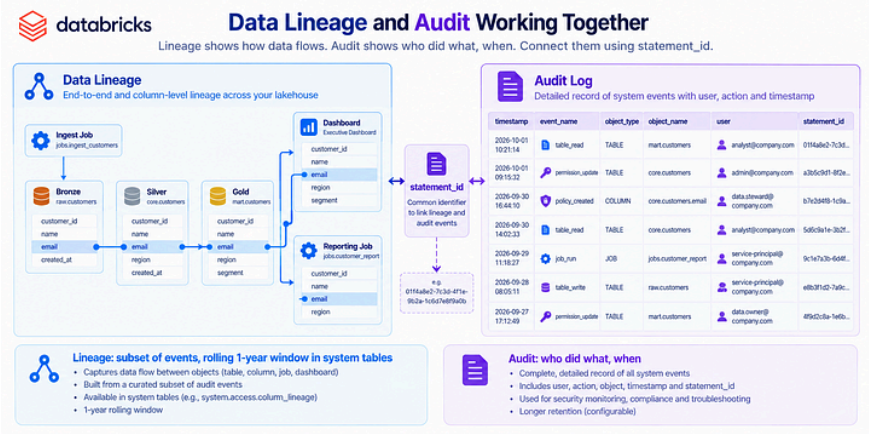
Lineage is the map of how data flows: which tables feed which tables, and which jobs, notebooks and dashboards touch them.

Why lineage matters

Use	Example
Impact analysis	"If I change this column, what breaks?"
Root-cause analysis	"This dashboard is wrong, which upstream table changed?" (builds on Level 5 tracing)
Compliance	"Show every place customer email data flows"
Trust	"Where did this metric come from?"
Cleanup	"Which tables does nobody read?"

What Unity Catalog captures

Unity Catalog lineage is captured automatically for queries run on supported compute, down to the column level, and covers tables, notebooks, jobs, pipelines and dashboards. You can see it in Catalog Explorer, through an API, and through system tables.



Querying lineage with system tables

The docs list two lineage system tables, `system.access.table_lineage` and `system.access.column_lineage`. Each record in the table lineage table is a read or write event on a table or path. Useful columns include `source_table_full_name`, `target_table_full_name`, `entity_type`, `created_by`, `event_time` and `statement_id`.

What reads from a given table (downstream impact):

```
SELECT DISTINCT
    target_table_full_name,
    entity_type,
    created_by
FROM system.access.table_lineage
WHERE source_table_full_name = 'prod.silver.cust
  AND target_table_full_name IS NOT NULL
  AND event_date >= current_date() - 90;
```

What feeds a given table (upstream sources):

```
SELECT DISTINCT
    source_table_full_name,
```

```
    entity_type
FROM system.access.table_lineage
WHERE target_table_full_name = 'prod.gold.daily_
  AND source_table_full_name IS NOT NULL
  AND event_date >= current_date() - 90;
```

Where does a PII column flow: the column lineage table works the same way at column level. Confirm its column names with `DESCRIBE system.access.column_lineage` in your workspace before building on it.

Limitations you must know before promising anything to an auditor

The docs are explicit [2]:

- Lineage tables hold a subset of read and write events. “Records are only emitted when lineage can be inferred.”
- System tables keep a rolling 1-year window. For longer history, use Catalog Explorer or the lineage API, which the docs say retain lineage captured after September 1, 2025 indefinitely.
- Reads through a view record the underlying base table with `direct_access = false`, so filter on that column when you want direct vs indirect dependencies.

So lineage is excellent for engineering and good evidence for compliance, but not a guarantee that every data movement is captured. Don’t tell an auditor it is complete. Say what it covers and what it doesn’t, and combine it with audit logs.

. . .

Step 7: Audit Logging—The Evidence Trail

Lineage tells you how data moved. Audit logs tell you who did what, and when.

The `system.access.audit` system table records account and workspace events, including data access and permission changes. Column names have been stable (`event_time` , `service_name` , `action_name` , `user_identity` , `request_params` , `response` , `source_ip_address`), but confirm them with `DESCRIBE` in your workspace.

Questions you should be able to answer with a query

Who changed permissions in the last week?

```
SELECT
    event_time,
    user_identity.email AS actor,
    action_name,
    request_params
FROM system.access.audit
WHERE event_date >= current_date() - 7
  AND service_name = 'unityCatalog'
  AND action_name IN ('updatePermissions', 'crea
ORDER BY event_time DESC;
```

Action names vary. Run `SELECT DISTINCT action_name FROM system.access.audit WHERE service_name = 'unityCatalog'` first to see what your account actually emits, then build the filter from real values.

Who accessed a sensitive table?

Combine audit events with query history, and use the lineage `statement_id` as the join key to the query history system table. Together they connect a person, a statement and a table.

Who is hitting data they never used before? This is a good anomaly-detection candidate, reusing your Level 5 methods: baseline each user's normal tables, and alert on novel access to Restricted data.

Make audit useful

1. Retain longer than the default if compliance requires it. Copy audit data into your own governed table on a schedule.
2. Alert on high-risk events: permission grants on Restricted schemas, policy deletions, new external locations, access from unusual locations.
3. Review periodically. Quarterly access reviews (“does this person still need this?”) are a standard audit control.
4. Protect the logs. Few people should be able to modify or delete audit data.

. . .

Step 8: GDPR and the Right to Be Forgotten on a Lakehouse

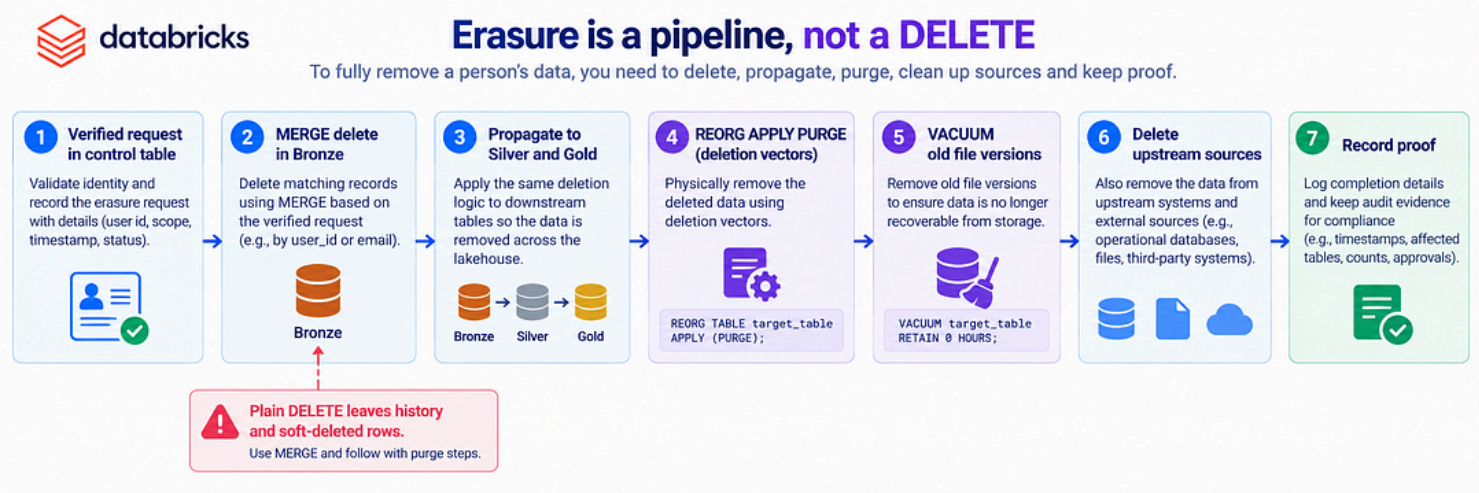
If you work for a European employer, this section will come up in interviews and in real life. (This is engineering guidance, not legal advice. Your legal and privacy teams define the obligations.)

GDPR and CCPA require that companies permanently and completely delete a person’s PII on request, within a specified period such as one calendar month. That sounds like a simple `DELETE`. On a lakehouse it isn't.

Why a plain DELETE isn’t enough

1. Delta keeps history. Old file versions stay in storage for time travel until `VACUUM` removes them.
2. Deletion vectors only mark rows as deleted. For tables with deletion vectors enabled, you must also run `REORG TABLE ... APPLY (PURGE)` to permanently remove the underlying records. This applies to Delta tables, materialized views and streaming tables.
3. Data lives in many layers. Bronze, Silver, Gold, exports, caches.

4. Data lives outside Delta. Kafka topics, landing files and source databases are in scope too.



The blueprint Databricks recommends

From the Databricks GDPR guidance [3]:

1. Delete in Bronze first, driven by a scheduled job that reads a table of deletion requests, then propagate deletions to Silver and Gold.
2. Use `MERGE` for bulk deletes against a control table.
3. Maintain tables so historical files are physically removed.
4. Delete upstream in queues, cloud storage and source systems.
5. Prefer complete deletion over obfuscation. Pseudonymization or masking may not remove re-identification risk, so erasure is the safest option.

A deletion workflow

```
-- Control table: one row per verified erasure r
-- gdpr_control_table(user_id, requested_at, prc
```

```
-- 1. Delete from Bronze using the control table
MERGE INTO prod.bronze.users AS t
USING (SELECT user_id FROM prod.governance.gdpr_
       WHERE processed_at IS NULL) AS s
ON t.user_id = s.user_id
WHEN MATCHED THEN DELETE;

-- 2. If deletion vectors are enabled, physicall
REORG TABLE prod.bronze.users APPLY (PURGE);

-- 3. Remove old file versions beyond retention
VACUUM prod.bronze.users;
```

Repeat for every downstream table that holds the person's data (or let your incremental pipelines propagate deletions, if they handle deletes). Then mark requests processed and keep proof.

Retention knobs to understand

Setting	Controls	Default
`delta.logRetentionDuration`	How long table history (the log) is kept	30 days
`delta.deletedFileRetentionDuration`	How long `VACUUM` keeps removed files	7 days

Until `VACUUM` runs past the retention window, "deleted" PII can still exist in older file versions and be recovered with time travel. For tables with PII, pick retention values deliberately and don't drop them below safe levels without understanding long-running readers and streams. The docs recommend Predictive Optimization for managed tables, or scheduled `VACUUM` otherwise, so that this maintenance isn't forgotten.

Design choices that make erasure cheap

- Use surrogate keys. If you key tables by `user_id` rather than email, you can delete the PII table and leave non-PII data intact.

- Centralize PII. One `customers_pii` table joined by `user_id` is far easier to erase from than PII scattered across 60 tables.
- Keep a deletion audit trail. Record request ID, tables touched and completion time. This is your compliance evidence.
- Remember backups, exports and ML features. Models trained on personal data and exported files are part of the conversation with your privacy team.

Related GDPR concepts engineers should know

Principle	Engineering implication
Data minimization	Don't ingest or retain fields you don't need
Purpose limitation	Access policies tied to purpose, not just role
Storage limitation	Retention policies and scheduled deletion
Access and portability requests	Ability to find all data for one person quickly
Data residency	Region choices for catalogs, storage and workspaces
Breach notification	Audit logs and alerting that let you scope an incident

Step 9: Secure Data Sharing

Sooner or later someone needs to share data outside the team, or the company. The worst answer is a CSV export by email.

Options, from safest to riskiest

Method	Use when	Governance
Grant access inside Unity Catalog	Internal consumers	Full: policies, audit, lineage
Delta Sharing	External partners, other platforms, other clouds	Share live data without copying it; access is revocable and logged
Clean rooms	Joint analysis without exposing raw data	Controlled collaboration
File exports	Last resort	Governance ends at the export

Delta Sharing Basics

```
CREATE SHARE partner_acme_share;

ALTER SHARE partner_acme_share
  ADD TABLE prod.gold.partner_sales_summary;

CREATE RECIPIENT acme_corp;  -- configure authen

GRANT SELECT ON SHARE partner_acme_share TO RECI
```

Principles:

- Share curated Gold views, not raw tables. A partner should see a purpose-built dataset.
- Combine with row-level restrictions so each partner sees only their own rows.
- Time-box access and review shares regularly.
- Log and review share activity.

Check the current Delta Sharing docs for the supported recipient types, and for the exact syntax for your edition.

. . .

Step 10: Governance Operating Model—People and Process

Technology enforces policy. People define it. A platform with perfect tools and no operating model still fails.

Roles

Role	Responsibility
Data owner	Accountable for a domain's data and who may access it
Data steward	Keeps definitions, classification and quality standards current
Platform / security admin	Operates Unity Catalog, identity integration and policy tooling
Data engineer	Implements pipelines that respect classification and retention
Privacy / compliance officer	Interprets regulation, approves retention and deletion approaches
Consumer	Requests access and uses data within policy

The access request lifecycle

1. Consumer requests access to a dataset (self-service catalog or ticket)
2. Data owner approves, based on purpose and classification
3. Access is granted to a group, with an expiry where applicable
4. Access is logged and reviewed on a cadence
5. Access is removed when no longer needed

Governance cadence

Cadence	Activity
Continuous	Policies enforce automatically; alerts on high-risk events
Monthly	Review new Restricted data, untagged columns, new external shares
Quarterly	Access recertification, with owners confirming who still needs access
Annually / on audit	Evidence packs: policies, access reviews, deletion logs, incident drills

Tie governance to earlier levels

- Level 4: CI/CD deploys policies as code, so access rules are versioned and reviewed like any other change
- Level 5: monitor governance health (untagged sensitive columns, policy failures, unusual access)
- Level 6: the same tags that drive access control also drive cost attribution

. . .

Step 11: A Worked Example—Securing a Customer Analytics Platform

Situation: a European retailer's `customers` and `orders` tables feed dashboards for EMEA and APAC teams. The data includes emails, phone numbers and addresses. A new customer-support team wants access, and legal asks for a deletion process.

Step 1: Structure. Environment catalogs (`prod`) with `bronze` , `silver` , `gold` schemas. Bronze is limited to the ingestion service principal and the data engineering group. Analysts get `SELECT` only on Gold.

Step 2: Identity. All access goes to groups synced from the identity provider: `analysts_emea` , `analysts_apac` , `support_agents` , `pii_readers` . Pipelines run as service principals. Table owners are groups.

Step 3: Classify. A discovery scan plus steward review tags `email` , `phone` and `address` columns with `data_class` values. Governed tags restrict who can assign them.

Step 4: Policies.

- One ABAC column mask policy masks every column tagged `pii_*` for everyone except `pii_readers`
- A row filter limits regional analysts to their region
- Support agents get a purpose-limited Gold view and masked PII, with unmasking only through a documented escalation

New tables ingested next quarter inherit protection if their columns are tagged. No one attaches masks by hand.

Step 5: Lineage and audit. Lineage shows exactly which Gold tables and dashboards depend on `silver.customers` , so the team can assess the impact of every change. Audit alerts fire on permission changes to Restricted schemas and on large reads by unusual users.

Step 6: Erasure. Legal's request process writes verified requests to a control table. A scheduled job deletes from Bronze via `MERGE`, propagates to Silver and Gold, runs `REORG ... APPLY (PURGE)` and `VACUUM`, and records completion. Upstream landing files are cleaned by a separate job.

Step 7: Prove it. For the next audit, the team produces:

- The policy definitions (in Git)
- A quarterly access review record
- A query showing masked output for a non-privileged test user
- A deletion log with timestamps
- A lineage export for the customer table

Result: access requests drop from days to hours for standard datasets, new tables are protected by default, and "did we delete it?" has a documented answer.

. . .

Step 12: Governance Anti-Patterns

Anti-pattern	Why it hurts	Better approach
Granting to individuals	Doesn't scale; orphaned access	Groups synced from the IdP
Pipelines running as personal accounts	Breaks when people leave; unclear audit trail	Service principals
Object owned by an individual	Orphaned when they leave	Group or service principal ownership
Attaching masks table by table	Inconsistent, forgotten on new tables	ABAC policies driven by tags
Open tagging rights	Anyone can downgrade sensitivity	Governed tags with controlled assignment
Protecting tables but not storage paths	Bypass via direct file access	Lock down external locations and credentials
`DELETE` and calling it GDPR compliant	History and deletion vectors keep the data	Purge, `VACUUM`, upstream deletes, proof
Obfuscation instead of erasure for RTBF	Re-identification risk remains	Delete, unless legal says otherwise
Exports to share data	Governance ends at the file	Delta Sharing or governed access
Treating lineage as complete	Captures a subset of events, 1-year retention in system tables	State limits; combine with audit logs
One big admin group	Everyone can do everything	Separate duties; least privilege
Governance as a one-time project	Decays within months	Continuous policy plus quarterly reviews
Never testing as a restricted user	Masks and filters may not work as you think	Verify with test identities

Step 13: Interview Questions at Level 7 (Governance, Security & Lineage)

Question	What a strong answer includes
How would you design access control for a multi-team lakehouse?	Catalog/schema hierarchy, groups from IdP, least privilege, service principals, grants at schema level, group ownership
Row filters vs column masks?	Filters limit rows, masks transform column values; both enforced on the table, not per tool
Why ABAC over per-table masks?	One tag-driven policy covers many tables and future ones; consistency; central policy, scalable
What are governed tags and why do they matter?	Tags with controlled assignment; attributes that policies evaluate; weak tag control undermines policy
How do you implement GDPR right to be forgotten?	Delete from Bronze via control table and `MERGE`, propagate downstream, `REORG ... APPLY (PURGE)` for deletion vectors, `VACUUM`, upstream deletes, audit trail
Why isn't a Delta `DELETE` enough?	Time travel history and deletion vectors retain data until purged and vacuumed
What's lineage useful for?	Impact analysis, root cause, compliance, cleanup; mention limits (subset of events, 1-year in system tables)
How would you answer "who accessed this table last month?"	Audit log plus query history, joined via statement ID; retention considerations
How do you share data with an external partner?	Delta Sharing of curated, filtered data; avoid exports; time-box and audit
Why run pipelines as service principals?	Stability, least privilege, clean audit trail
How do you prevent bypass of table security?	Restrict external locations, storage credentials and direct storage access
How do you keep governance from slowing teams down?	Self-service for standard access, automated policies, clear owners, group-based requests
How do you prove controls to an auditor?	Policies as code, access review records, test outputs, audit/lineage queries, deletion logs

Step 14: A Governance Rollout Path

Don’t try to do it all at once.

Month 1: Foundations

- Define catalog and schema structure by environment and layer
- Sync groups from the identity provider
- Move pipelines to service principals
- Set group ownership on key objects
- Lock down storage credentials and external locations

Month 2: Classification and protection

- Define the tag taxonomy and governed tags
- Classify the highest-risk tables first
- Mask PII with column masks or ABAC policies
- Add row-level security where regional or tenant separation is needed

Month 3: Visibility

- Build lineage and impact-analysis queries
- Copy audit logs to a governed, longer-retention table
- Alert on permission changes and unusual access to Restricted data
- Run the first access recertification

Month 4: Compliance workflows

- Build the erasure workflow with control table, purge and vacuum
- Document retention per data class
- Define sharing standards (Delta Sharing, no exports)
- Put policies in Git with CI/CD

Month 5+: Maturity

- Automate classification scans
- Report governance KPIs (see below)
- Drill an incident: “scope a data exposure using audit and lineage”
- Extend to ML assets and AI features

Governance KPIs

- Percentage of sensitive columns classified and tagged
- Percentage of tables with owners
- Number of direct (non-group) grants
- Time to fulfill a standard access request
- Time to complete an erasure request
- Access recertification completion rate

. . .

Conclusion

Databricks Level 7 (Governance, Security & Lineage) is where a data platform earns the right to hold important data.

At Databricks Level 6 (Performance & Cost Optimization), you made the system lean. At Databricks Level 7, you make it safe to rely on, and safe to be questioned about.

The core ideas:

- Model access with the Unity Catalog hierarchy, grant to groups, and run automation as service principals
- Enforce fine-grained security on the data itself with row filters and column masks
- Scale it with ABAC, where tags drive policy, and new data is protected by default
- Use lineage to see impact and origin, and know its limits
- Use audit logs as your evidence trail
- Treat erasure as a pipeline: delete, purge, vacuum, propagate, prove
- Share through governed channels instead of exports

The best governance is invisible to people doing legitimate work and impossible to ignore for people who aren't. It gives you a platform where "who can see this?" and "did we delete it?" are queries, not investigations.